Decision Tree Algorithm

1. Introduction

A **Decision Tree** is a **supervised machine learning algorithm** used for **classification and regression** problems.

It works like a **flowchart**:

- Each internal node → a decision (condition)
- Each branch → outcome of the decision
- Each leaf node → final prediction

---

2. Structure of Decision Tree

A decision tree consists of:

1. **Root Node**
    - Topmost node
    - Represents the entire dataset
2. **Decision Nodes**
    - Nodes where data is split based on a condition
3. **Leaf Nodes**
    - Final output (class label or value)
4. **Branches**
    - Represent outcomes of decisions

---

3. Working of Decision Tree

The algorithm works in the following steps:

Step 1: Select Best Feature

- Choose the feature that best splits the data

Step 2: Split the Dataset

- Divide data based on a condition (e.g., Age > 30)

Step 3: Repeat Recursively

- Continue splitting until:
    - Data becomes pure OR
    - Stopping condition is reached

Step 4: Make Prediction

- Final leaf node gives output

Decision Trees for different purposes 👇

---

1. Classification (Pass / Fail)

👉 Purpose: Predict category (Yes/No)

Is study hours > 5?
├── Yes → Pass
└── No → Fail

✅ Used in exams, loan approval, spam detection

---

2. Regression (Predict Marks)

👉 Purpose: Predict a numeric value

Is study hours > 6?
├── Yes → Marks = 85
└── No → Marks = 55

✅ Used for price prediction, marks prediction

---

3. Medical Diagnosis

👉 Purpose: Predict disease

Is fever = Yes?
├── Yes → Is cough = Yes?
│ ├── Yes → Flu
│ └── No → Viral Fever
└── No → Healthy

✅ Used in healthcare decision systems

---

4. Loan Approval

👉 Purpose: Approve or reject loan

Is income > 50,000?
├── Yes → Is credit score > 700?
│ ├── Yes → Loan Approved
│ └── No → Loan Rejected
└── No → Loan Rejected

✅ Used in banking and finance

---

5. Customer Purchase Prediction

👉 Purpose: Will customer buy or not?

Is age < 30?
├── Yes → Is income high?
│ ├── Yes → Buy
│ └── No → Not Buy
└── No → Buy

✅ Used in marketing and sales

---

6. Weather-Based Decision (Play or Not)

👉 Purpose: Decide action

Is weather sunny?
├── Yes → Is humidity high?
│ ├── Yes → Don't Play
│ └── No → Play
└── No → Play

✅ Classic ML example

---

7. Fraud Detection

👉 Purpose: Detect fraud transaction

Is transaction amount > 50,000?
├── Yes → Is location unusual?
│ ├── Yes → Fraud
│ └── No → Normal
└── No → Normal

✅ Used in banking security

All Decision Trees follow the same pattern:

👉 **Ask a question → Split → Repeat → Final decision**

🔷 Example (Linear and Non-Linear data)

Problem: Predict whether a student will **pass or fail** based on:

- Hours studied
- Sleep hours

🟦 Case 1: Linear Relationship (Logistic Regression works well)

👉 Rule is simple:

- More study hours → higher chance of passing
- Less study → fail

This forms a **straight-line boundary**.

Example:

- Study > 5 hours → Pass
- Study ≤ 5 hours → Fail

✅ Here, **Logistic Regression** works perfectly.

🟥 Case 2: Non-Linear Relationship (Decision Tree works better)

👉 Now imagine a more realistic situation:

- If study > 5 AND sleep > 6 → Pass
- If study > 5 BUT sleep < 4 → Fail (too tired)
- If study < 3 → Fail
- If study between 3–5 → depends on sleep

👉 This creates a **complex, zig-zag pattern**, not a straight line.

🌳 How Decision Tree Handles It

A Decision Tree will break it like this:

Is study > 5?
├── Yes → Is sleep > 6?
│ ├── Yes → Pass
│ └── No → Fail
└── No → Is study < 3?
├── Yes → Fail
└── Depends on sleep

✅ It handles **conditions and rules naturally**
✅ No need for straight-line separation

Key Difference

Feature	Logistic Regression	Decision Tree
Boundary	Straight line	Complex shapes
Works best when	Data is linear	Data is non-linear
Logic	Mathematical equation	If-else rules
Example	Pass if score > 50	Pass if (score + sleep + mood conditions)

👉 So when data is messy, rule-based, or non-linear → **use Decision Tree**
👉 When data follows a clear linear trend → **use Logistic Regression**

4. Important Metrics

🔷 Entropy

Entropy is a measure of the "messiness" or chaos in your basket.

- Entropy = 0 (Pure): Your basket contains 10 apples and 0 oranges. It is perfectly organized.
- Entropy = 1 (Impure): Your basket contains 5 apples and 5 oranges. It is a total mess; you have no idea what you'll pick out.
- Measures **impurity (disorder)**
- Range: 0 (pure) to 1 (impure)

$$H(S) = -\sum p_i \log_2 p_i$$

- $H(S)$: The Entropy of a set S.
- $\sum$: The "Sigma" symbol, meaning we **sum** (add up) the results for every category in our set.

- p_i: The **probability** (or proportion) of a specific category i. For example, if 3 out of 10 fruits are apples, p{apple} = 0.3.
- $\log_2 p_i$: The **logarithm base 2**. This is used because in computer science, information is measured in "bits" (0s and 1s).
- **The Negative Sign (-)**: Because probabilities are decimals (between 0 and 1), their logs are always negative. The minus sign at the front flips the result back to a positive number.

Example

Imagine you have a basket with 4 fruits:

- 2 apples → probability = 2/4 = 0.5
- 2 oranges → probability = 2/4 = 0.5

Since both fruits are equally present (50–50), the mixture is completely random. So, the entropy should be 1.

Now calculate:

- For apples:
  $0.5 \times \log_2(0.5) = 0.5 \times (-1) = -0.5$
- For oranges:
  $0.5 \times \log_2(0.5) = 0.5 \times (-1) = -0.5$

Add them:
$-0.5 + (-0.5) = -1$

Apply the negative sign:
Entropy = $-(-1) =$ **1**

Why this makes sense:

- If all fruits are the same (for example, only apples), then probability = 1 and $\log_2(1) = 0$ → entropy = 0 (no uncertainty).
- If fruits are evenly mixed, uncertainty is maximum → entropy becomes highest (1).

In short:
Entropy measures how much uncertainty or randomness is present in the data before

you observe it.

🔷 Information Gain

Information Gain measures how much clutter you cleared away by splitting the fruit.

- Example: If your main basket has an entropy of 1.0, and after splitting it into two bowls, the average entropy drops to 0.2, your Information Gain is 0.8.
- Reduction in entropy after split

$$IG = Entropy(parent) - Entropy(children)$$

👉 Higher IG = better split

🔷 Gini Index

The Gini Index is like a shortcut for Entropy. It calculates the probability of picking a fruit and mislabeling it.

- Gini = 0: Perfect purity (you'll never guess wrong).
- Gini = 0.5: Maximum impurity (it's a 50/50 coin toss).
- Comparison: While Entropy uses complex math (log functions), Gini uses simple squares. This makes it faster for computers to calculate. Lower is better.
- Measures impurity using probability

$$Gini = 1 - \sum p_i^2$$

👉 Lower Gini = better split

🔷 Gain Ratio

Gain Ratio is the "fairness" filter for Information Gain.

Standard Information Gain has a flaw: it loves attributes with many unique values. If you tried to split your fruit by a "Serial Number" printed on the skin, every fruit would

be in its own "perfectly pure" bowl. This looks great on paper but is useless for predicting new fruit.

Gain Ratio penalizes these "too-specific" splits. It ensures the tree picks useful categories (like color or shape) rather than just memorizing unique IDs.

- Improves Information Gain by reducing bias

$$Gain\ Ratio = \frac{IG}{Split\ Information}$$

---

5. Types of Decision Trees

1. **Classification Tree**
    - Output: Category (Yes/No)
    - Example: Pass/Fail prediction
2. **Regression Tree**
    - Output: Numeric value
    - Example: Salary prediction

---

6. Algorithms of Decision Tree

Algorithm	Technique Used
ID3	Information Gain
C4.5	Gain Ratio
CART	Gini Index

1. ID3 (Iterative Dichotomiser 3)

👉 **Idea:**
Builds a tree by choosing the feature that gives the **highest Information Gain** (best split).

👉 **Key Points:**

- Uses **Entropy & Information Gain**
- Works mainly with **categorical data**
- Can **overfit** easily
- Does NOT handle missing values well

👉 **Simple View:**
"Choose the question that reduces randomness the most."

---

2. C4.5 (Improved ID3)

👉 **Idea:**
An improved version of ID3 that fixes its problems.

👉 **Key Improvements:**

- Uses **Gain Ratio** (instead of only Information Gain)
- Handles **continuous data** (like age, salary)
- Can handle **missing values**
- Includes **pruning** (reduces overfitting)

👉 **Simple View:**
"Smarter ID3 that avoids bad splits and handles real-world data."

---

3. CART (Classification and Regression Trees)

👉 **Idea:**
Builds binary trees (only **2 branches**) using a different metric.

👉 **Key Points:**

- Uses **Gini Index** (not entropy)
- Supports:
    - **Classification** (Yes/No)

    - **Regression** (numeric output)
- Always creates **binary splits**
- Widely used in practice (e.g., sklearn)

👉 **Simple View:**
"Practical tree that splits data into two parts each time."

Quick Comparison

Feature	**ID3**	**C4.5**	**CART**
Metric	Information Gain	Gain Ratio	Gini Index
Data Type	Categorical	Both	Both
Handle Missing Values	❌ No	✓ Yes	✓ Yes
Tree Type	Multi-branch	Multi-branch	Binary only
Pruning	❌ No	✓ Yes	✓ Yes

---

7. Advantages

✓ 1. Easy to Understand and Visualize

Decision Trees look like a **flowchart**, so anyone can understand them.

Example:

Is study > 5?
├── Yes → Pass
└── No → Fail

✅ Even a non-technical person can follow this
✅ No complex math needed

✓2. Requires Less Data Preprocessing

You don’t need to do much cleaning or transformation before using data.

Example:

- No need to normalize values (like scaling marks from 0–1)
- Can handle missing values (in many implementations)

✅ Saves time
✅ Works directly on raw data

✓ 3. Works with Both Numerical and Categorical Data

It can handle:

- Numbers (marks, age)
- Categories (Pass/Fail, Yes/No, Male/Female)

Example:

Is gender = Male?
Is marks > 50?

✅ No need to convert everything into numbers manually
✅ Flexible with different data types

✓ 4. Fast and Interpretable

- Training is usually quick
- Predictions are fast
- You can clearly **see why** a decision was made

Example:
If a student fails, you can trace:

- Study < 3 → Fail

---

8. Disadvantages

❌ 1. Overfitting (Tree becomes too complex)

The tree learns **too much detail**, even noise (random errors) in the data.

Example:
Suppose:

- Most students who study > 5 hours pass
- But 1 student studied 6 hours and still failed

A complex tree might create a rule like:
"If study = 6 AND name = Rahul → Fail"

This is **too specific** and useless for new data.

Problem:

- Works perfectly on training data
- Fails on new/unseen data

❌ 2. Sensitive to Small Data Changes

A small change in data can create a **completely different tree**.

Example:
Dataset:

- 10 students → tree says: "Study > 5 → Pass"

Now add just **1 extra student** who:

- studied 6 hours but failed

The tree may change rule to:

- "Check sleep also"

Entire structure of tree changes due to small change.

Problem:

- Model becomes **unstable**
- Not reliable sometimes

❌ 3. Can Become Biased

The tree may favour one class or feature more than others.

Example:
Suppose dataset:

- 90 students passed
- 10 students failed

Tree may learn:

- "Always predict Pass"

Accuracy = 90%, but it's useless for detecting failures.

Another case:

- Feature like "Marks" dominates
- Tree ignores "Sleep" or "Health"

Problem:

- Ignores minority class
- Gives **unfair predictions**

---

9. Overfitting and Underfitting

- **Overfitting**
    - Tree is too deep
    - Memorizes data
    - Poor performance on new data
- **Underfitting**
    - Tree is too simple
    - Misses patterns

👉 Solution:

- Pruning (cut unnecessary branches)
- Limit tree depth

---

10. Real-World Applications

- Medical diagnosis (disease prediction)
- Loan approval systems
- Fraud detection
- Customer churn prediction
- Stock market decision making

Conclusion

Decision Tree is a **powerful and simple algorithm** that:

- Mimics human decision-making
- Is easy to interpret
- Is widely used in real-world problems

However, it must be properly controlled to avoid overfitting.

Comparison of Decision Tree with Other Algorithms

🎯 (A) Classification Algorithms Comparison

Feature	Decision Tree	Logistic Regression	K-Nearest Neighbors (KNN)	Naive Bayes	Support Vector Machine (SVM)
Type	Tree-based	Linear model	Distance-based	Probabilistic	Margin-based
Working Idea	Splits data using rules	Finds linear boundary	Uses nearest points	Uses probability (Bayes theorem)	Finds best separating boundary
Interpretability	⭐ Very High	High	Low	Medium	Low
Accuracy	Medium	Medium	Medium–High	Medium	High
Speed (Training)	Fast	Fast	Very Fast	Very Fast	Slow
Speed (Prediction)	Fast	Fast	Slow	Fast	Medium
Handles Non-linearity	✅ Yes	❌ No (basic)	✅ Yes	❌ Limited	✅ Yes
Feature Scaling Needed	❌ No	✅ Yes	✅ Yes	❌ No	✅ Yes
Overfitting Risk	High	Low	Medium	Low	Medium
Best Use Case	Rule-based	Linear	Small	Text	Complex

Feature	Decision Tree	Logistic Regression	K-Nearest Neighbors (KNN)	Naive Bayes	Support Vector Machine (SVM)
	decisions	problems	datasets	classification	boundaries

🎯 (B) Regression Algorithms Comparison

Feature	Decision Tree	Linear Regression	Random Forest	Gradient Boosting
Type	Tree-based	Linear	Ensemble (many trees)	Ensemble (boosting)
Accuracy	Medium	Low–Medium	High	Very High
Interpretability	High	Very High	Low	Low
Handles Non-linearity	✅ Yes	❌ No	✅ Yes	✅ Yes
Overfitting	High	Low	Low	Medium
Training Speed	Fast	Very Fast	Medium	Slow
Prediction Speed	Fast	Very Fast	Medium	Medium
Complexity	Simple	Very Simple	Complex	Very Complex
Best Use Case	Simple decision rules	Linear trends	High accuracy tasks	Competitions / high performance

Key Differences:

🔷 Decision Tree vs Logistic Regression

- Decision Tree → rule-based

- Logistic Regression → equation-based

👉 Use Tree when data is **non-linear**

🔷 **Decision Tree vs KNN**

- Tree → learns model
- KNN → stores data and compares

👉 KNN is **slow in prediction**

🔷 **Decision Tree vs Naive Bayes**

- Tree → uses splits
- Naive Bayes → uses probability

👉 NB works best for **text data**

🔷 **Decision Tree vs SVM**

- Tree → easy to understand
- SVM → complex but more accurate

🔷 **Decision Tree vs Random Forest**

- Tree → single model (weak)
- Random Forest → many trees (strong)

👉 Random Forest reduces overfitting ✅

 # Classification Tree using Gini Index

```
from sklearn.tree import DecisionTreeClassifier

# Simple data: [Study Hours, Sleep Hours]
```

```
X = [[1,5], [2,6], [5,7], [6,8]]   # Inputs
y = [0, 0, 1, 1]                   # 0 = Fail, 1 = Pass

# Create model (Gini is default)
model = DecisionTreeClassifier()

# Train model
model.fit(X, y)

# Predict new student
print("Prediction:", model.predict([[4,6]]))  # Output: Pass (1)
```

 # Classification Tree using Entropy

```
from sklearn.tree import DecisionTreeClassifier

X = [[1,5], [2,6], [5,7], [6,8]]
y = [0, 0, 1, 1]

# Use entropy
model = DecisionTreeClassifier(criterion='entropy')

model.fit(X, y)

print("Prediction:", model.predict([[3,6]]))
```

 # Regression Tree

```
from sklearn.tree import DecisionTreeRegressor

# Data: [Experience in years]
X = [[1], [2], [3], [4], [5]]

# Salary (in lakhs)
y = [2, 3, 5, 7, 9]

# Create model
model = DecisionTreeRegressor()

# Train
```

```
model.fit(X, y)

# Predict salary for 3.5 years experience
print("Predicted Salary:", model.predict([[3.5]]))
```

 # Visualizing Decision Tree

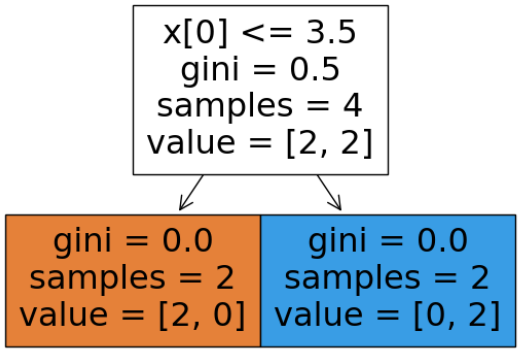
```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

X = [[1,5], [2,6], [5,7], [6,8]]
y = [0, 0, 1, 1]

model = DecisionTreeClassifier()
model.fit(X, y)

plt.figure()
plot_tree(model, filled=True)
plt.show()
```

Output:



What This Ouput Shows

It is a **Decision Tree used for classification**.

- It splits data based on a condition
- Then gives a final decision (class)

Top Box (Root Node)

x[0] <= 3.5
gini = 0.5
samples = 4
value = [2, 2]
Meaning:
1. x[0] <= 3.5
👉 First feature (column 0) is being checked
👉 Rule:

- If ≤ 3.5 → go LEFT
- If > 3.5 → go RIGHT

---

2. gini = 0.5
👉 Data is **mixed (impure)**
👉 0.5 = maximum impurity for 2 classes

---

3. samples = 4
👉 Total **4 data points** at this node

---

4. value = [2, 2]
👉 Class distribution:

- 2 belong to Class 0
- 2 belong to Class 1

👉 Perfectly mixed

⬅️ **Left Node**
gini = 0.0
samples = 2
value = [2, 0]
Meaning:

- 2 samples
- All belong to Class 0
- Pure node → gini = 0

👉 Final decision = **Class 0**

➡️ **Right Node**
gini = 0.0
samples = 2
value = [0, 2]
Meaning:

- 2 samples
- All belong to Class 1
- Pure node → gini = 0

👉 Final decision = **Class 1**

Complete Logic (Tree Rule)

👉 The model is basically doing this:

If x[0] <= 3.5 → Class 0

Else → Class 1

Key Concepts in This Output

Term	Meaning
x[0]	First feature
gini	Impurity (0 = pure)
samples	Number of data points
value	Class distribution

Final Understanding

- Root node is **confused (mixed data)**
- Tree splits data using a condition
- After split → both sides become **pure (perfect classification)**
